

Loyola Finance Group

Antes "Loyola Capital Group"

Especificación funcional, modelo de datos y diagramas — MVP

Documento generado a partir del mockup de diseño

1. Flujo operativo

SaaS de financiamiento y cobranza para negocios de servicios (demo: Loyola Capital, construcción — Roofing, Air Conditioning, Home Improvement).

1. **Administración** registra un Lead: contacto, categoría de servicio, foto de DNI y de recibo.
2. **Venta:** un vendedor elige un lead disponible (activo → asignado), agenda una visita y completa los datos del cliente (asignado → en_proceso). El vendedor no elabora la cotización ni ve el SSN/ITN.
3. **Operaciones y crédito (Admin):** elabora la cotización (en Roofing, mide escuadras con herramienta), captura el SSN/ITN, consulta el score en Experian (registro histórico) y decide manualmente aprobar o rechazar (máximo USD 25,000).
4. **Contrato:** firma, 50% de downpayment verificado por un admin, modalidad (12 a 120 cuotas, tasa mensual, medio de cobro vía Stripe) y comisión del vendedor (se devenga al firmar, se descuenta del monto entregado).
5. **Construcción:** LC ejecuta la obra y registra avances/entregas.
6. **Cobranza automática (Stripe):** cobro por token con clave de idempotencia en cada vencimiento; reintenta ante fallos.
7. **Mora:** 5 días de gracia, luego 5% fijo + 1%/semana adicional, tope 10%. Cada cargo es una fila independiente en el ledger.

2. Roles

Acción	Admin	Vendedor	Cliente
Registrar / ver leads	✓	✓ (los suyos)	×
Elegir lead disponible	×	✓	×
Elaborar cotización	✓	×	×
Ver / capturar SSN-ITN	✓	×	×
Aprobar o rechazar crédito	✓	×	×
Ver dashboard de préstamos (todos)	✓	×	×
Ver su propio préstamo (portal)	×	×	✓
Configurar negocio	✓	×	×

3. Categorías de servicio

Categoría	Unidad de medida	Notas
Roofing	Escuadras	Se mide con herramienta dedicada
Air Conditioning	Toneladas	
Home Improvement	Horas	Subtipos: Baño, Cocina, Recamara, Pisos, Fences, Ventana, Roofing, Pintura

Las categorías y sus campos viven en `categoria_servicio` / `campo_servicio` — agregar una categoría nueva no requiere cambios de esquema.

4. Máquina de estados

Lead: activo → asignado (vendedor lo elige) → en_proceso (datos completos) → convertido (crédito aprobado) | descartado.

Cuota: pendiente → pagada (cobro exitoso) | vencida (pasó la fecha) → en_mora (pasan los días de gracia) → pagada (pago tardío, capital + mora).

Solicitud de crédito: pendiente → aprobado | rechazado (con motivo).

5. Secuencias clave

Aprobación de crédito

Vendedor completa datos → Sistema notifica a Admin → Admin crea cotización → Admin solicita consulta de score → Sistema llama a Experian → Experian responde → Sistema guarda en consulta_score → Admin aprueba/rechaza → Sistema notifica al cliente.

Cobro automático

Cron diario revisa cuotas que vencen hoy → Sistema crea cargo en Stripe con clave de idempotencia → Stripe intenta cobrar → Stripe envía webhook (charge.succeeded / charge.failed) → Sistema inserta fila en movimiento_pago → si exitoso, marca Cuota pagada; si falla, reintenta; si se agotan los reintentos, marca vencida/en_mora e inserta un movimiento tipo Mora.

6. Arquitectura de componentes

- **Supabase/Postgres:** fuente de verdad (leads, clientes, contratos, préstamos, ledger).
- **Tu app:** orquesta todo — lee/escrbe en Postgres, llama a Stripe y Experian, sirve el portal del cliente.
- **Stripe:** tokeniza medios de pago, cobra por API, envía webhooks de resultado.
- **Experian:** consulta de score vía API, bajo autorización del cliente.
- *A futuro:* HubSpot (sincroniza Leads/Clientes) y un proveedor de firma electrónica sobre el Contrato.

7. Ledger de pagos (regla de oro)

La tabla `movimiento_pago` es inmutable: solo se hace INSERT, nunca UPDATE ni DELETE. El saldo de una cuota se calcula sumando sus movimientos, no leyendo un campo de estado editable.

8. Fuera de alcance del MVP

- CRM de clientes (integrar HubSpot free más adelante vía API).
- Firma electrónica (DocuSign / Dropbox Sign, cuando se justifique el costo).

9. Ya construido en este MVP

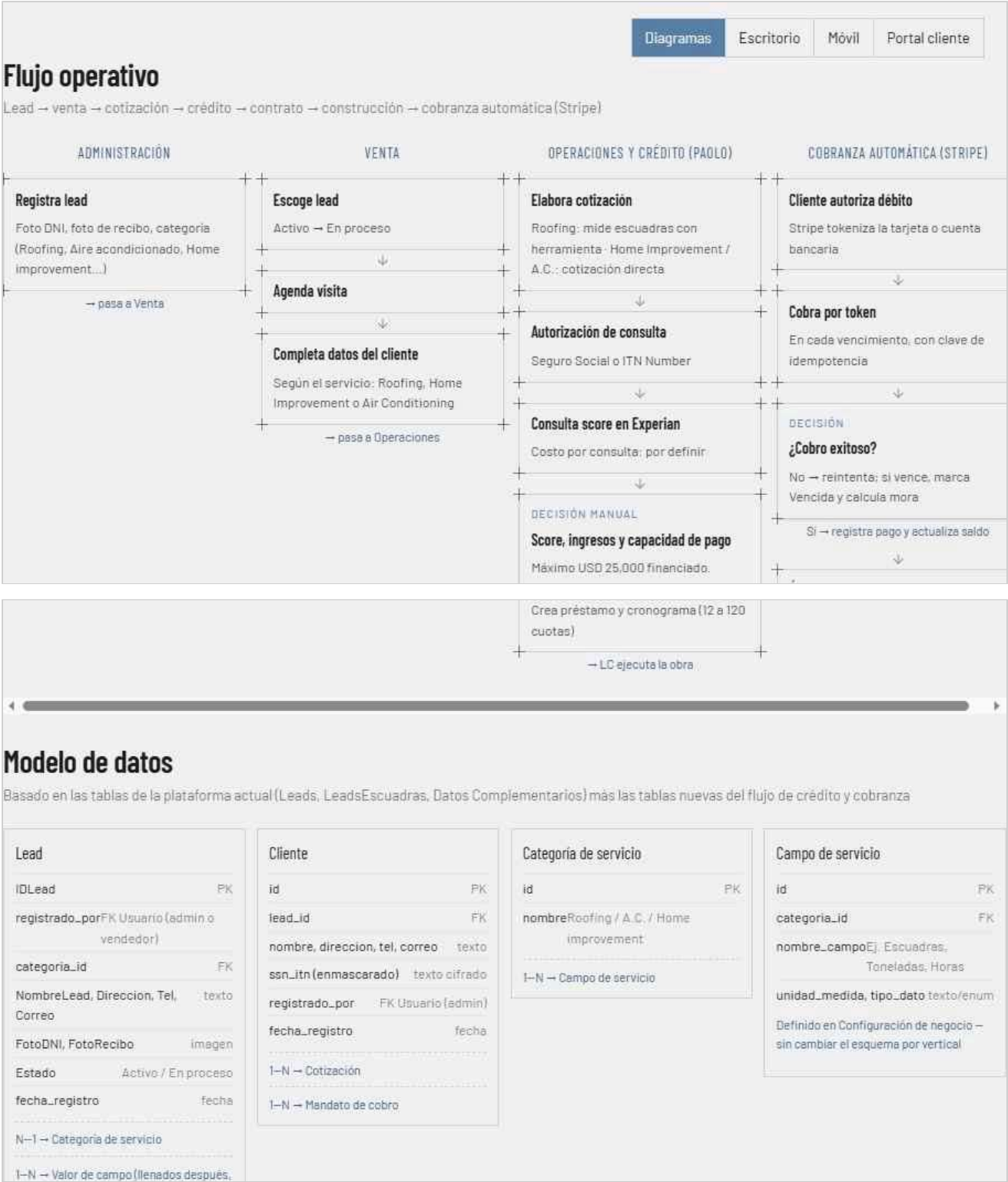
- Portal del cliente (escritorio y móvil).
- Cobranza automática vía Stripe.
- Comisión de vendedor con ciclo de vida propio.
- Consulta de score histórica (nunca se sobrescribe).

10. Próximo paso técnico

1. Correr `schema.sql` en Supabase.

2. Construir primero el CRUD simple: Lead → Cliente → Cotización.
3. Integrar Experian y Stripe una vez el CRUD esté probado.
4. Portal del cliente al final, sobre las mismas tablas.

Anexo A — Diagramas



id	PK
lead_id	FK
campo_servicio_id	FK
valor	texto/decimal
registrado_por	FK Usuario
fecha_registro	fecha
1-1 → Cotización (una vez completos)	

id	PK
cliente_id, elaborada_por	FK (Admin)
monto total	decimal (suma de ítems)
requiere_financiamiento	bool
fecha_registro	fecha
1-N → Ítem de cotización	
1-1 → Solicitud de crédito	

id	FK
cotizacion_id	FK
descripcion	Materiales, mano de obra...
cantidad, precio_unitario	decimal
subtotal	decimal

id	PK
cotizacion_id	FK
decisión, motivo	enum/texto
decidido_por	FK Usuario (Paolo)
fecha_registro	fecha
1-N → Consulta de score (histórico; usa la más reciente)	
1-1 → Contrato	

Consulta de score (Experian)	
id	PK
solicitud_id, cliente_id	FK
score, ingresos, capacidad_pago	decimal
registrado_por	FK Usuario
fecha_registro	fecha (inmutable)
Cada consulta queda como registro fijo, nunca se sobrescribe – permite reevaluar el score con el tiempo	

Contrato	
id	PK
solicitud_id	FK
n° cuotas (12-120), tasa	int/decimal
vendedor_id	FK
downpayment_pagado	bool
registrado_por	FK Usuario
fecha_registro	fecha
1-1 → Comisión de vendedor	
1-1 → Préstamo	

Comisión de vendedor	
id	PK
contrato_id, vendedor_id	FK
porcentaje, monto	decimal
estado	Devengada/Pagada/Revertida
fecha_registro	fecha
Se devenga al firmar el contrato y se descuenta del monto entregado	

Préstamo	
id	PK
contrato_id	FK
monto financiado	decimal
estado	enum
verificado_por	FK Usuario (downpayment)
fecha_registro	fecha
1-N → Cuota	
N-1 → Mandato de cobro	

<table><tr><td>estado</td><td>Pendiente/Pagada/Vencida</td></tr><tr><td colspan="2">1-N → Movimiento de pago (ledger)</td></tr></table>	estado	Pendiente/Pagada/Vencida	1-N → Movimiento de pago (ledger)		<table><tr><td>monto_interes</td><td></td></tr><tr><td>stripe_event_id,</td><td>texto</td></tr><tr><td>clave_idempotencia</td><td></td></tr><tr><td>resultado, fecha_registro</td><td>enum/fecha</td></tr><tr><td colspan="2">Inmutable – nunca se edita, solo se agregan filas nuevas (ver nota debajo)</td></tr></table>	monto_interes		stripe_event_id,	texto	clave_idempotencia		resultado, fecha_registro	enum/fecha	Inmutable – nunca se edita, solo se agregan filas nuevas (ver nota debajo)		<table><tr><td>fecha_registro</td><td>fecha</td></tr></table>	fecha_registro	fecha
estado	Pendiente/Pagada/Vencida																	
1-N → Movimiento de pago (ledger)																		
monto_interes																		
stripe_event_id,	texto																	
clave_idempotencia																		
resultado, fecha_registro	enum/fecha																	
Inmutable – nunca se edita, solo se agregan filas nuevas (ver nota debajo)																		
fecha_registro	fecha																	

¿QUÉ ES EL LEDGER DE PAGOS? Stripe puede enviar un webhook por cada evento real (cobro exitoso, fallo, reintento, reembolso, disputa). Cada uno se guarda como una fila nueva en "Movimiento de pago" – nunca se edita ni se borra. El saldo de una cuota se calcula sumando esas filas; no confiando en un campo "estado" que alguien podría cambiar a mano. Esto te da: auditoría completa, reconciliación con el dashboard de Stripe, y una forma limpia de registrar reembolsos o disputas cuando ocurran.	SOLUCIÓN PRÁCTICA PARA LA MORA (A AFINAR CON EL TIEMPO) Regla implementada: 5 días de gracia tras el vencimiento, luego un cargo fijo de mora del 5% de la cuota, más 1% adicional por cada semana que siga vencida, con un tope de 10%. Cada cargo se registra como su propia fila "Mora" en el ledger – así puedes ajustar el % o los días de gracia después sin migrar datos, y cada cargo queda explicado si el cliente lo disputa.
--	---

Máquina de estados

Transiciones válidas – nada debería mover un registro fuera de estas flechas

Cuota

Pendiente

→

Pagada

(cobro exitoso en el ledger)

Pendiente

→

Vencida

(pasó la fecha sin cobro exitoso)

Lead

Activo

→

Asignado

(un vendedor lo elige)

Asignado

→

En proceso

(datos del cliente completados)

Orden temporal de los dos flujos con más riesgo: aprobación de crédito y cobro automático

Aprobación de crédito 1. Vendedor → Sistema: completa datos del cliente 2. Sistema → Admin: lead listo para cotizar 3. Admin → Sistema: crea cotización (con items) 4. Admin → Sistema: solicita consulta de score 5. Sistema → Experian: envía SSN/ITN + datos 6. Experian → Sistema: responde score, ingresos, capacidad 7. Sistema → Admin: muestra resultado (guarda en Consulta de score) 8. Admin → Sistema: aprueba o rechaza (con motivo) 9. Sistema → Cliente: notifica siguiente paso (contrato/downpayment)	Cobro automático (Stripe) 1. Sistema (cron diario) revisa cuotas que vencen hoy 2. Sistema → Stripe: crea cargo con clave de idempotencia 3. Stripe → Cliente: intenta cobrar tarjeta/cuenta 4. Stripe → Sistema: webhook charge.succeeded / charge.failed 5. Sistema → Ledger: inserta fila en movimiento_pago 6. Sistema → Cuota: si exitoso, marca Pagada; si falla, programa reintento 7. (tras reintentos fallidos) Sistema → Cuota: marca Vencida/En mora + inserta movimiento tipo Mora 8. Sistema → Cliente/Admin: notifica el resultado
---	---

Arquitectura de componentes

Qué sistemas hablan entre sí y cómo

SUPABASE / POSTGRES Fuente de verdad: leads, clientes, contratos, préstamos, ledger	TU APP Frontend + backend Orquesta todo: lee/escribe en Postgres, llama a Stripe y Experian, sirve el portal	STRIPE Tokeniza medios de pago, cobra por API, envía webhooks de resultado	EXPERIAN Consulta de score via API, bajo autorización del cliente
	sirve el portal		

A futuro: HubSpot (sincroniza Leads/Clientes via API, sin ser fuente de verdad del crédito) y un proveedor de firma electrónica sobre el Contrato.

Matriz de permisos

ACCIÓN	ADMIN	VENDEDOR	CLIENTE
Registrar / ver leads	✓	✓ (los suyos)	×
Elegir lead disponible	×	✓	×
Elaborar cotización	✓	×	×
Ver / capturar SSN-ITN	✓	×	×
Aprobar o rechazar crédito	✓	×	×
Ver dashboard de préstamos (todos)	✓	×	×
Ver su propio préstamo (portal)	×	×	✓
Configurar negocio (categorías, comisiones)	✓	×	×

Fuera de alcance del MVP

×	CRM de clientes	×	Firma electrónica
---	-----------------	---	-------------------

Anexo B — Esquema SQL (schema.sql)

```
-- Loyola Finance Group – esquema inicial (Postgres / Supabase)
-- Ver el archivo docs/schema.sql en el repositorio para la versión ejecutable completa.
-- Resumen de tablas: usuario, categoria_servicio, campo_servicio, lead,
-- valor_campo_lead, cliente, cotizacion, item_cotizacion, consulta_score,
-- solicitud_credito, contrato, comision_vendedor, mandato_cobro,
-- prestamo, cuota, movimiento_pago (ledger immutable).
```